

A Data-Fact-Driven Visual Analysis Method for Generative Time Series Data Augmentation

Fei Lyu, Guangze Deng, Zhuo Tang, *Member, IEEE*, Kenli Li, *Senior Member, IEEE*,
Changjian Chen, *Member, IEEE*

Abstract—The success of time series deep learning models relies on large amounts of high-quality data for training. However, in many critical scenarios, such as stock market crash prediction, collecting sufficient and high-quality time series data is challenging. Generative time series data augmentation has become a promising solution to address data scarcity. Nevertheless, direct application of these methods often leads to degraded performance of downstream models due to low-quality generated samples. In this paper, we develop TimeShaper, a visual analysis method for involving humans in the generative time series data augmentation process to improve the quality of generated time series samples. We develop fact-based visualization supported by a representative data fact extraction method and a co-layout method to facilitate the exploration of both the original and generated time series samples. Based on the exploration, we further design a constraint visualization with a unified design principle, which helps users create or refine constraints to guide the generation process for better performance. The effectiveness of our proposed methods is demonstrated through quantitative evaluation and a case study.

Index Terms—Time series, generative data augmentation, data fact, constraint visualization

I. INTRODUCTION

TIME series data play a fundamental role in various real-world applications, ranging from financial forecasting [1] and healthcare monitoring [2] to industrial fault diagnosis [3]. These tasks typically require large amounts of data to train high-performance deep learning models [4]. However, in many critical scenarios, such as stock market crash prediction, it is difficult to collect sufficient and high-quality time series data [5]. In recent years, the rapid development of generative models has given rise to a promising solution for tackling data scarcity: **generative data augmentation** [6]. Rather than relying solely on limited original samples, such methods first learn the underlying data distribution from them and impose pre-defined constraints to produce new samples. Then, the combination of original and generated samples is used to enrich the training dataset, thereby facilitating more robust training of downstream models ([7], [8]).

Although generative data augmentation methods have achieved remarkable success in generating more training time series samples, directly applying them remains problematic. Our experimental results show that, in some cases, the performance of downstream models decreases with the amount

of generated time series data. This degradation is caused by the generation of many low-quality samples, which stems from the gap between the pre-defined constraints and real-world applications. First, the limited number of pre-defined constraints often leads to unexpected generated samples. For example, generating new samples for temperature time series may produce unrealistically high temperatures (Fig. 1 Case 1). Second, due to the complexity of the constraints, not all the constraints can be satisfied, leading to ambiguous samples. For example, when generating new samples based on the traffic volume data, the generated samples exhibit counterintuitive patterns, where traffic volume peaks occur during extreme weather (Fig. 1 Case 2).

An effective way to reduce low-quality time series samples is to involve humans to provide proper constraints and edit wrong ones. However, providing/editing constraints is non-trivial because of two challenges. First, the generated time series samples need to be examined to identify the unexpected ones. However, the generated time series samples are not only large in size but also typically cover long time spans with a high density of data points, resulting in an immense data scale. This scale poses a critical challenge: manual inspection of low-quality samples within such massive time series data is both time-consuming and highly error-prone for humans. Therefore, a tool that enables efficient exploration and analysis is needed. Second, when identifying unexpected samples, humans struggle to explicitly define their prior knowledge as constraints that can be integrated into the generative model to guide the generation process.

To address these challenges, we develop TimeShaper, a visual analysis system designed to enable users to 1) efficiently explore both the original and generated time series samples and 2) intuitively guide the data generation process. To facilitate in-depth exploration of both the original and generated time series samples, we adopt the concept of data fact, which effectively characterizes the information within time series data [9]. By examining data facts instead of individual data points within time series samples, users can not only obtain deeper insights into the data but also reduce time and effort in the exploration process. Motivated by this, we propose a fact-based visualization. To help identify key data facts among the numerous time series samples, this visualization extracts representative facts from them, which can be formulated as a subset selection problem with temporal coverage constraints. Then, to facilitate the comparison, this visualization uses our proposed data fact co-layout method to align similar representative data facts of the original and generated samples in adjacent

F. Lyu, G. Deng, Z. Tang, K. Li, and C. Chen are with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. E-mail: {feilv, gzdeng, ztang, lkl, changjianchen}@hnu.edu.cn. C. Chen is the corresponding author.

Manuscript received xxx. xx, 2026; revised xxx. xx, 2026.

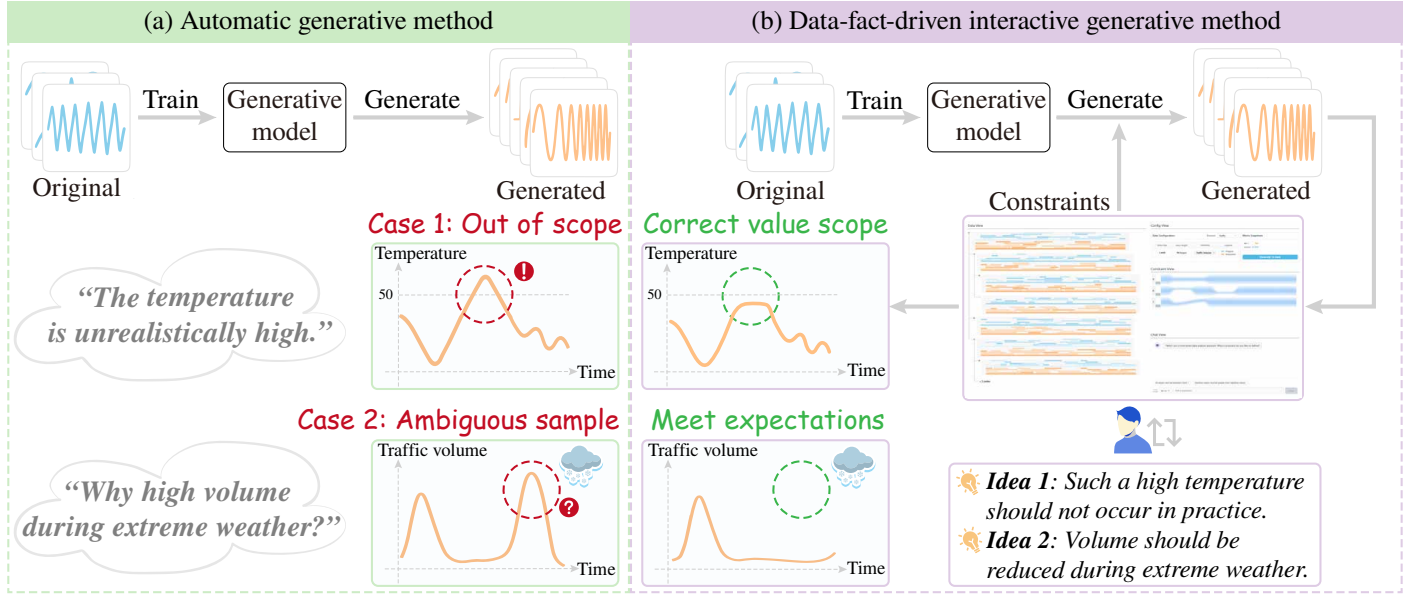


Fig. 1. The comparison between the automatic and the data-fact-driven interactive generative methods. (a) The automatic generative method is uncontrollable, which leads to low-quality time series samples. (b) Our proposed data-fact-driven interactive generative method allows users to impose constraints to control the generation process, resulting in correct time series samples.

positions. With the fact-based visualization, users explore and compare the original and generated time series samples and identify the low-quality ones. With the low-quality generated samples, users analyze how constraints are validated, propose new constraints, and perform corresponding edits. To ease this process, we develop a constraint visualization that follows a unified design principle, which is not only intuitive to users but also capable of covering diverse constraint types. The effectiveness of the fact-based visualization method and the constraint visualization method is demonstrated through quantitative evaluation and a case study.

In summary, the contributions of this work include:

- A **fact-based visualization** supported by a representative data fact extraction method and a data fact co-layout method to extract and visualize data facts for efficient time series sample exploration and comparison.
- A **constraint visualization** with a unified design principle to facilitate constraint construction and refinement.
- A **visual analysis tool** that enables the user to interactively guide the time series data generation process.

II. RELATED WORK

Our work focuses on interactive visual analysis of the generative time series data. Existing work can be categorized into time series data exploration and time series data quality improvement.

A. Time series data exploration

Time series data exploration methods aim to help users understand temporal patterns, detect anomalies, and compare changes across different time series data. Based on whether they are directly applied in downstream tasks, these methods

can be classified into two categories: task-agnostic and task-aware exploration methods.

Task-agnostic exploration methods focus on general-purpose exploration and pattern discovery without targeting specific downstream tasks. Zhao *et al.* [10] introduced KronoMiner, an interactive analysis system that supports multi-foci navigation for pattern exploration and simultaneous manipulation of multiple time segments. Sips *et al.* [11] proposed a matrix-based visualization that allows users to uncover latent temporal structures. To improve scalability, [12] and [13] applied dimensionality reduction techniques to project time series onto a 2D plane for exploration. PSEUDO [14] employed locality-sensitive hashing to facilitate the extraction of visual patterns from large-scale time series data. Bernard *et al.* proposed IVESA [15], a system that integrates multiple linked views to support exploration of large-scale timestamped event sequences. To further save user effort, Visail [16] used reinforcement learning to autonomously generate visualization sequences that guide users during data exploration and reduce cognitive load. Liu *et al.* developed RelaQ [17], which enables time series querying through relation specifications. More recently, ChronoDeck [18] introduced hierarchical time series exploration across different levels of aggregation. TiVy [19] presented an algorithm that utilizes sequential patterns to summarize and visualize time series data. Huang *et al.* [20] proposed a transformer-based framework that learns interpretable shapelet-based signatures and an interactive interface to support exploration.

Despite their generality, these task-agnostic exploration methods often lack the domain-specific adaptations needed for specialized downstream tasks. Therefore, task-aware exploration methods are proposed.

Task-aware exploration methods are often tightly integrated with downstream tasks, such as time series forecasting

and anomaly detection. For time series forecasting tasks, Xu *et al.* [21] developed mTSeer, incorporating diverse visualizations and interactive tooltips to facilitate the exploration and evaluation of forecasting models. Yi *et al.* [22] combined time series forecasting models with decision tree regression to help users forecast power generation trends. Hao *et al.* [23] developed TimeTuner, which employs counterfactual explanations to enhance interpretability and exploration of temporal patterns. In addition, many studies ([24], [25], [26], [27], [28]) have been proposed for anomaly detection tasks. For example, Lv *et al.* [24] proposed a visual analysis system PEVis, which integrates multiple anomaly detection algorithms to help users uncover evolving anomaly patterns. Similarly, Omar *et al.* [25] introduced VIGMA, which supports personalized treatment and long-term rehabilitation monitoring through analyzing patient gait time series data. Beyond anomaly detection tasks, other methods have also been proposed for causal analysis. Deng *et al.* [29] introduced a compass-based visualization to explore causal relationships in urban time series data.

Although effective, these methods cannot be directly applied to time series data generation tasks, which require comparability between the generated and original data. To address this gap, we propose a fact-based time series data visualization method that supports exploration and comparative analysis of generated and original time series data.

B. Time series data quality improvement

Time series data quality improvement typically includes annotation quality improvement and intrinsic data quality improvement, such as missing value imputation, outlier detection, etc. In practice, existing methods for improving time series data quality can be classified based on whether they target annotations or the data itself.

Annotation quality improvement methods aim to help correct annotation errors and ensure annotation consistency for time series data. For example, Liu *et al.* [30] developed a visual analysis system called MTV, which supports a workflow for detecting, investigating, and annotating anomalies in multivariate time series through human-AI collaboration. In the context of video time series, VideoPro [31] enables scalable and consistent labeling of video frames by mining sequential patterns. Chen *et al.* [32] improved temporal annotation by propagating sparse labels and incorporated storytelling visualization to help users interactively correct localization and alignment errors. Tang *et al.* [33] proposed a learning-based method and developed a system to help users filter high-risk segments in the time series data.

Intrinsic data quality improvement methods focus on improving the inherent quality of time series data rather than solely refining annotations. Many visual analysis methods have been proposed to support this goal. TimeCleanser [34] combines automated quality checks with editable visualizations to enable interactive cleansing of time series data. Visplause [35] facilitates efficient quality assessment in large-scale time series data through plausibility checks and linked views. Yeon *et al.* [36] proposed a visual analysis framework that assists in missing value imputation within Bayesian networks for

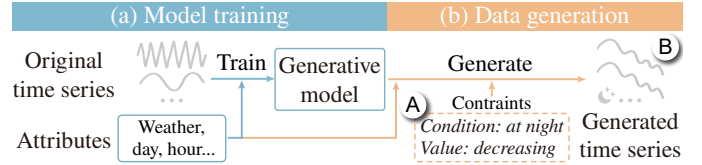


Fig. 2. Generative time series data augmentation consists of two main steps: (a) generative model training and (b) data generation.

time series data. KYE [37] offers predefined and customizable quality checks with visual exploration to help identify raw data quality issues, while [38] introduced a visual-interactive pipeline to support uncertainty-aware preprocessing for multivariate time series data.

These time series data quality improvement methods are effective for correction and refinement but fall short in data generation tasks, which typically require integrating prior domain knowledge (*e.g.*, domain-specific constraints). Existing methods lack an effective way to convert such knowledge into model constraints. To address this, we introduce a constraint visualization method that converts user knowledge directly into model constraints, and supports examining and refining these constraints to improve time series data generation.

III. BACKGROUND: TIME SERIES GENERATION

Recently, many methods ([6], [39]–[44]) have been proposed for generative time series data augmentation. According to the literature review, we summarize these methods into a common pipeline, as shown in Fig. 2, which consists of two steps: generative model training and data generation.

Generative model training. In the generative model training step, the original time series samples and their corresponding attributes (*e.g.*, temperature, day, and hour) are fed into a generative model (*e.g.*, diffusion models) for training (Fig. 2(a)). This model learns the distribution of the original time series samples conditioned on their attributes. In this paper, we employ Time Weaver [41] for training, which is a state-of-the-art diffusion-based conditional generative model tailored for time series generation with constraints.

Data generation. Once training is complete, the model can be used for generating time series samples. In the data generation step, we input the constraints into the model to generate new samples (Fig. 2(b)). The constraints consist of two components: condition and value (Fig. 2A). The condition specifies the expected attributes of the generated time series samples, and the value defines their expected pattern (*e.g.*, the value is decreasing). Then, the generative model produce new samples adhere to the given constraints (Fig. 2B).

IV. REQUIREMENT ANALYSIS

We developed this work in close collaboration with four machine learning experts (E1-E4). E1 and E2 are Ph.D. students. Both of them specialize in meteorological time series data, have extensive experience with generative models, and share a strong motivation to advance the field of time series. E3 is a researcher in an academic institute, and E4

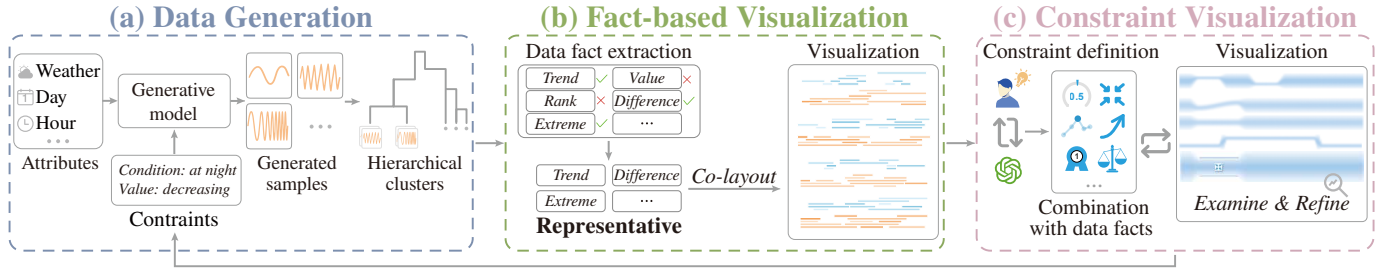


Fig. 3. Method overview: (a) given attributes and constraints, the generative model produces time series samples, which are then hierarchically clustered; (b) data facts from original and generated samples are extracted and visualized; (c) the constraints are defined with data facts and visualized for examination and refinement. The refined constraints are fed into the generative model to generate new samples.

is a machine learning engineer in a technology company. During their research, they found that limited time series data often constrain model performance, motivating them to apply generative techniques for data augmentation. However, they observed that many generated time series samples fall short of expectations. Consequently, they expressed a need for a more controllable time series data generation process. We collected the following requirements by conducting four semi-structured interviews with the experts, each of which lasted between 40 and 70 minutes.

R1: Efficiently exploring and comparing original and generated time series data at a large scale. Given the characteristics of time series data, such as long time spans and high data point density, all experts agreed that identifying problematic samples is extremely time-consuming, especially when dealing with large-scale generated time series samples. E2 mentioned, “It would be much more efficient for me if the summary information could be directly observed such that I do not need extensive data exploration.” In addition, E4 emphasized the importance of comparing the generated and the original samples. He stated, “Since generated samples are derived from the original dataset, an intuitive comparison between the two enables us to clearly understand the difference, such as numerical difference and trend change, without the need for constantly switching between them.”

R2: Effectively and efficiently converting user knowledge into constraints to guide the generation process. When identifying low-quality generated time series samples, experts aim to integrate their domain knowledge into the model to guide the generation process and obtain more desirable samples. However, translating such expert knowledge into model constraints is often ineffective and inefficient. For example, E3 noted that when observing abnormal extreme values in the generated time series samples, substantial time and effort are required to examine when these extreme values occur and their magnitudes in order to determine appropriate constraints. Thus, there is a need for an effective and efficient way to convert expert knowledge into model constraints, thereby saving time and reducing manual effort.

R3: Examining the constraints applied to the generation process in an intuitive way. The experts pointed out that after converting their ideas into model constraints, it is necessary to verify how these constraints work in the generation process and whether these constraints meet ex-

pectations. If the constraints are inaccurate, applying them to the generation process can lead to unexpected issues and flawed outputs. For example, E1 said, “It always takes me a long time to verify the correctness and completeness of the model constraints, especially when determining where the constraints take effect.” To address this, the experts proposed that presenting constraints in an intuitive way would be an effective solution. E1 further suggested that enabling users to interactively modify these constraints could reduce the effort required for iterative refinement.

V. DESIGN OF TIMESHAPER

Based on the requirements, we develop TimeShaper to facilitate visual analysis for generative time series data augmentation. Fig. 3 shows its overview. Given the attributes and constraints, a set of time series samples is generated (Fig. 3(a)). Then, the original and generated samples are organized into a hierarchy [45]. For each cluster, representative data facts are extracted from the generated and original samples within that cluster and displayed using a co-layout method (Fig. 3(b), **R1**). Users can explore this visualization to identify low-quality samples. During exploration, users can express their ideas to define constraints with data facts (Fig. 3(c), **R2**). Furthermore, to examine whether the constraints meet expectations, users can display them for examination and refinement (Fig. 3(c), **R3**). The refined constraints are fed into the generative model to generate new samples. This process is repeated until the generated data quality and downstream model performance meet their requirements.

A. Fact-based Time Series Data Visualization

According to the expert interview, a key requirement (**R1**) is the efficient exploration and comparison of original and generated time series samples at a large scale. A straightforward way is to visualize them side by side using line charts [46]. However, this method may lead to significant visual clutter when handling large-scale generated samples (Fig. 4(a)). Moreover, exploring and identifying problematic samples requires users to summarize key insights about them, which imposes additional cognitive burdens. To tackle these issues, inspired by [9], we adopt the concept of data fact, which refers to numerical or statistical results derived from the data. Data facts can effectively summarize and characterize

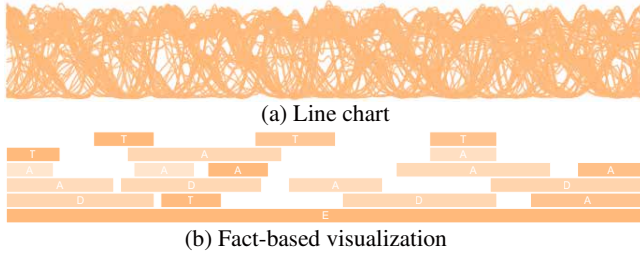


Fig. 4. A comparison example between the (a) line chart and (b) our proposed fact-based visualization.

the information contained in time series data, which help users obtain an overview of temporal patterns instead of inspecting the time series individually, while also significantly reducing the visual clutter (Fig. 4(b)).

Since data facts are typically numerous, to reduce the user efforts in analysis, we propose a **representative data fact extraction method**. Based on the extracted data facts, we utilize fact-based time series data visualization to present them. Fig. 5(a) shows the fact-based time series visualization and illustrates the visual encoding of data facts. Specifically, each horizontal bar represents a data fact and encodes four parts: the letter inside the bar indicates the fact type; the color hue distinguishes original (in blue) and generated (in orange) samples; the color intensity represents the normalized value among facts of the same type (darker meaning higher); and the bar length indicates the covered time interval. These encodings help users understand what type of pattern occurs, where it

occurs, how long it lasts, and whether its value differs between original and generated samples. To facilitate the comparison between the data facts of the original and generated samples, we developed a co-layout algorithm that aligns data facts of the same type and similar time intervals across the two groups.

1) *Representative Data Fact Extraction*: According to [9], there are eleven types of data facts. However, based on our analysis and discussion with the experts, not all the data facts are suitable to be constraints for the time series generation models. For example, the data fact *proportion* describes the proportional composition of different types of time series samples. However, since each sample is generated independently, it is difficult to constrain the model to produce samples that collectively satisfy such a data fact. Therefore, through in-depth discussions with the experts, we identified six types that are suitable for translation into constraints supported by the time series generative models. The experts further confirmed that these six types cover the vast majority of practical cases. These types are as follows: 1) *value*, which represent concrete values; 2) *aggregation*, which represent summary statistics (e.g., mean values); 3) *trend*, which captures the temporal tendency of time series; 4) *rank*, which indicates the relative ordering of time intervals within a sample; 5) *difference*, which denotes internal variations within the same time series; 6) *extreme*, which refers to the maximum and minimum values.

Following the definition of data facts in prior work [9], we extract data facts from each time series sample by enumerating time intervals and computing the corresponding statistics for each fact type. This strategy produces multiple candidate data



Fig. 5. TimeShaper comprises four main views: (a) a fact-based time series data visualization to show representative data facts for both original and generated time series samples; (b) a configuration panel to show data information and distance metric changes; (c) a constraint visualization to show and refine constraints; and (d) a chat panel for defining constraints via natural language.

facts for each sample, along with saliency scores, as well as the attributes calculated based on timestamp statistics. However, since the number of extracted data facts is quite large, it is still difficult for users to explore all of them. Therefore, it is necessary to select a representative subset of data facts that can reflect the overview of the data. This subset of data facts is expected to 1) be as salient as possible; 2) represent, as much as possible, the data facts of all time series samples; 3) cover all time intervals as comprehensively as possible; and 4) remain as small as possible to reduce users' exploration effort. To achieve these goals, we propose a subset selection algorithm with temporal coverage constraints to better extract representative data facts.

Problem setting. Let $\mathcal{T} = \{T_1, T_2, \dots, T_M\}$ be a collection of M time series samples, each consists of n values at n timestamps $[t_1, t_2, \dots, t_n]$. $\mathcal{F} = \{F_1, F_2, \dots, F_M\}$ is the data facts extracted from $\mathcal{T}$. $F_i = \{f_{i1}, f_{i2}, \dots, f_{iP_i}\}$ denotes the data facts extracted from T_i , and $U_i = \{u_{i1}, u_{i2}, \dots, u_{iP_i}\}$ is the time intervals associated with the data facts. P_i is the number of data facts of T_i . Each data fact f_{ij} correspond to a time interval $u_{ij} = [t_s, t_e]$ ($1 \leq s < e \leq n$).

Our objective is to select a compact and salient subset of data facts $F^{\text{rep}} = \{f_1^{\text{rep}}, f_2^{\text{rep}}, \dots\}$ that can well represent all the data facts $\mathcal{F}$. With $U^{\text{rep}} = \{u_1^{\text{rep}}, u_2^{\text{rep}}, \dots\}$ denoting the associated time intervals of these representative data facts, this objective can be achieved by maximizing the following cost function:

$$\begin{aligned} \max_{F^{\text{rep}}} \quad & \sum_{i=1} \text{saliency}(f_i^{\text{rep}}) - \lambda_1 \sum_{i=1} \sum_{f \in Q(u_i^{\text{rep}})} \text{diff}(f, f_i^{\text{rep}}) \\ & - \lambda_2 \|F^{\text{rep}}\|, \\ \text{s.t.} \quad & [t_1, t_n] \subseteq \bigcup_i u_i^{\text{rep}}. \end{aligned} \quad (1)$$

The first term encourages the saliency of the selected data facts. The second term encourages the representativeness of the selected data facts. The third term minimizes the number of selected data facts. The constraint ensures the selected data fact covers all time intervals. λ_1 and λ_2 control the trade-off between these terms. In this paper, we set $\lambda_1 = \lambda_2 = 1$ to achieve a balance between representativeness and brevity. Since the calculation of the third term and the constraints is straightforward, we first focus on elaborating the first and second terms.

The *first term* is the sum of the saliency scores ($\text{saliency}(\cdot)$) of the selected data facts. Following [9], the saliency score of a data fact is the sum of its normalized value and its interval length. The value is min-max normalized across all data facts of the same type.

The *second term* is the total dissimilarity between the selected data facts and the unselected data facts represented by them. $Q(u_i^{\text{rep}})$ represents all the unselected data facts in $\mathcal{F}$ sharing the same time interval u_i^{rep} . $\text{diff}(f_1, f_2)$ is defined as the dissimilarity between two data facts f_1 and f_2 . Based on the components of a data fact, inspired by [16], the $\text{diff}(\cdot, \cdot)$ is computed as the normalized sum of four parts: the temporal difference, calculated as $1 - \text{IoU}$ to capture discrepancies in time intervals; the type difference, which is 0 for identical types and 1 otherwise to encode semantic difference; the score

difference, representing the absolute difference in saliency scores; and the attribute difference, derived by comparing the dominant values of corresponding attributes.

Solution. Eq. (1) is a subset selection algorithm with temporal coverage constraints, which is NP-hard [47]. To reduce computational cost, we proposed a local-to-global strategy to solve it. The local selection step extracts a salient and compact subset of data facts for each time series sample independently, reducing the scale of candidate facts. The global selection step selects a global set of representative data facts for all time series samples.

Step 1: local selection. Given a single time series T_i , we select a subset F_i^{saliency} of its candidate data facts F_i that achieves the highest total saliency score. To avoid information loss, the selected data facts must together cover all timestamps. In addition, to avoid redundant content and keep the selected set compact, we limit the temporal overlap between any two consecutive fact intervals to no more than 2 timestamps. Let U_i^{saliency} be the time intervals of the selected data facts. For a single time series T_i , we formalize this local selection task as:

$$\begin{aligned} \max \quad & \sum_{f \in F_i^{\text{saliency}}} \text{saliency}(f), \\ \text{s.t.} \quad & [t_0, t_n] \subseteq \bigcup_{u \in U_i^{\text{saliency}}} u, \\ & |u \cap v| \leq 2, \quad \forall u, v \in U_i^{\text{saliency}}, u \neq v. \end{aligned} \quad (2)$$

This optimization can be solved by dynamic programming.

Step 2: global selection. Since the saliency data facts are most likely kept in F_i^{saliency} during Step 1, the global selection only selects the global set of representative data facts from the union of all F_i^{saliency} . This is formally achieved by optimizing the following objective:

$$\begin{aligned} \min_{F^{\text{rep}}} \quad & \lambda_1 \sum_{i=1} \sum_{f \in Q(u_i^{\text{rep}})} \text{diff}(f, f_i^{\text{rep}}) + \lambda_2 \|F^{\text{rep}}\|, \\ \text{s.t.} \quad & [t_1, t_n] \subseteq \bigcup_i u_i^{\text{rep}}. \end{aligned} \quad (3)$$

Here, $Q(u_i^{\text{rep}})$ represents all the unselected data facts in $\{F_1^{\text{saliency}}, F_2^{\text{saliency}}, \dots, F_M^{\text{saliency}}\}$ sharing the same time interval u_i^{rep} . Although the candidate set has been significantly reduced after the local selection, this optimization remains NP-hard. Thus, we adopt a greedy algorithm to obtain an approximate solution. We iteratively select the data fact that covers the maximum number of uncovered data facts; this process repeats until all data facts are covered. If any time intervals remain uncovered, we select data facts that intersect with the uncovered time intervals and have the longest intervals, repeating this step until all time intervals are fully covered.

2) *Data Fact Co-Layout:* After selecting the most representative data facts, we need to conduct efficient comparative analysis between the original and generated samples. Given that each data fact has a specific time interval, a common way is to encode each data fact as a horizontal bar [48], where the left and right ends of the bar respectively represent the start and end of the corresponding time interval. To avoid wasting space, a packing algorithm such as the Bin-Packing algorithm [49] can be used to optimize the positions of the

bars. Then, the two bar charts for the original and generated time series samples can be placed side by side for comparison. However, this strategy will increase users' cognitive load during comparison analysis for two issues. First, within the bar chart of the original/generated samples, data facts of the same type may be scattered across multiple rows (e.g., Fig. 6A), making it difficult to analyze similar data facts simultaneously. Second, between the bar charts of the original and generated samples, users often need to compare the data facts of the same type that occur within similar time intervals. However, the corresponding bars of such data facts may not be placed at the same rows (e.g., Fig. 6B), which hinders quick identification and direct comparison.

To address these issues, we propose a data fact co-layout algorithm. This algorithm organizes the data facts by two principles: 1) data facts of the same type are placed in adjacent rows within a bar chart of the original/generated samples, and 2) data facts of the same type that occur within similar time intervals are placed in similar rows across the bar charts of the original and generated samples. Let $F^{\text{ori}} = \{f_1^{\text{ori}}, f_2^{\text{ori}} \dots\}$ and $F^{\text{gen}} = \{f_1^{\text{gen}}, f_2^{\text{gen}} \dots\}$ be the representative data facts of the original and generated time series samples. Their associated time intervals are $U^{\text{ori}} = \{u_1^{\text{ori}}, u_2^{\text{ori}} \dots\}$ and $U^{\text{gen}} = \{u_1^{\text{gen}}, u_2^{\text{gen}} \dots\}$. Since the horizontal position of a representative data fact is determined by its associated time interval, only its vertical position needs to be optimized. The vertical position of a representative data fact f_i^{ori} (f_i^{gen}) can be denoted by l_i^{ori} (l_i^{gen}), indicating that it lies in the l_i^{ori} -th (l_i^{gen} -th) row. With these notations, the proposed algorithm aim to optimize the following objective:

$$\begin{aligned}
 \min_{L^{\text{ori}}, L^{\text{gen}}} \quad & \max_{i,j} (l_i^{\text{ori}} + l_j^{\text{gen}}) \\
 & + \omega_1 \sum_{w \in \{\text{ori}, \text{gen}\}} \sum_{i,j} \mathbb{1}_{[c(f_i^w)=c(f_j^w)]} |l_i^w - l_j^w| \\
 & + \omega_2 \sum_{(i,j) \in G} |l_i^{\text{ori}} - l_j^{\text{gen}}|, \quad (4) \\
 \text{s.t.} \quad & l_i^{\text{ori}} \neq l_j^{\text{ori}}, \quad \text{if } u_i^{\text{ori}} \cap u_j^{\text{ori}} \neq \emptyset, \\
 & l_i^{\text{gen}} \neq l_j^{\text{gen}}, \quad \text{if } u_i^{\text{gen}} \cap u_j^{\text{gen}} \neq \emptyset.
 \end{aligned}$$

The first term minimizes the total height of two bar charts, ensuring a compact visualization. The second term ensures that the data facts of the same type are placed closely. Here, $\mathbb{1}_{[c(f_i^w)=c(f_j^w)]}$ equals 1 if $c(f_i^w) = c(f_j^w)$; otherwise $\mathbb{1}_{[c(f_i^w)=c(f_j^w)]}$ equals 0, where $c(f_i^w)$ denotes the type of the data fact f_i^w . The third term facilitates alignment of comparable data facts between the two charts. Here, G represents the set of similar data fact pairs. For each original/generated data fact, the generated/original data fact with a $\text{diff}(\cdot, \cdot)$ value less than 0.5 is considered as a similar pair. Minimizing this term encourages similar data facts to be placed in the same row, enabling direct visual comparison. The constraint item is used to avoid time interval overlaps between data facts. The weights ω_1 and ω_2 balance the magnitude differences among the three terms and are determined by grid search. Fig. 6 presents a comparison between the direct layout and our proposed co-layout method. In the comparison result, our proposed co-layout method effectively clusters data facts of the same type

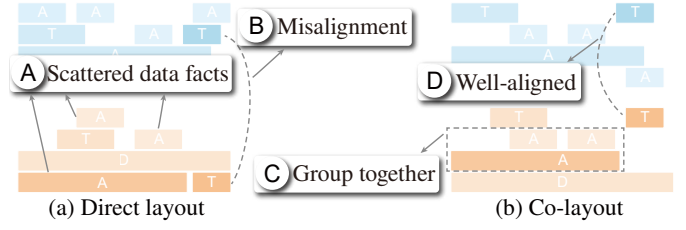


Fig. 6. The comparison between the (a) direct layout method and (b) our proposed co-layout method.

(Fig. 6C) and aligns similar data facts well (Fig. 6D). This allows users to quickly locate and compare related data facts, thereby saving efforts.

B. Constraint Visualization

When the low-quality generated samples are identified by users, converting their knowledge into model constraints is needed (R2). To this end, an LLM (e.g., GPT-4 [50]) is utilized as a parser to extract user knowledge and define constraints Fig. 5(d). However, directly using the constraints is problematic because they may contain errors. To tackle this issue, we develop a **constraint visualization** that follows a unified design principle to help users examine and refine the constraints (R3). The constraint visualization is shown in Fig. 5(c), where the rightmost bar represents the constraint displayed by our design principle. The leftmost bar is the constraint type, and the left bar shows the constraint violation ratio among all generated samples, with  indicating violation.

1) *Design Principles*: Due to the complexity of the constraints, directly presenting them in a clear and interpretable form is challenging [51]. To address this issue, we introduce a unified design principle to systematically represent these constraints. This principle ensures both clarity and interpretability of the constraint representation, while also allowing users to flexibly refine the constraint.

Visual encoding. A constraint comprises two components: a condition and a value. For example, in the constraint “the temperature on Monday is between 0 and 10”, the condition specifies the applicable scope (“Monday”), and the value defines the expected property of the time series (“temperature ranges from 0 to 10”). Accordingly, we split the visual encoding of constraints into two separate parts: condition and value, each encoded with independent visual designs. For conditions that are usually heterogeneous and difficult to symbolize, we represent them directly as textual tags instead of complex visual encoding. This avoids overloading the visual representation while preserving the key semantic information of the constraint. For the value that defines the characteristic that time series are expected to follow, we use a pair of prominent horizontal lines to explicitly mark the upper and lower bounds within each time interval, and a white dotted line to represent the mean value. To further illustrate the expected distribution of time series values within these bounds, inspired by [52], we apply a color intensity encoding that gradually diffuses from a concentrated center to faded boundaries. Such a design not only specifies valid value ranges

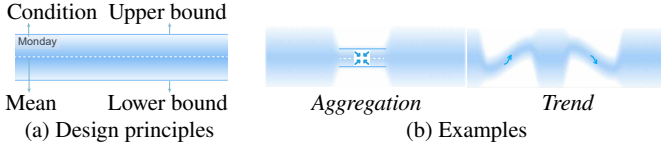


Fig. 7. Illustration of (a) our proposed unified design principles and (b) example constraint visualizations.

but also intuitively conveys the distribution that the constrained time series should follow. Fig. 7(a) shows our proposed design principles. In addition, as time is the fundamental dimension of time series data, every constraint must be explicitly anchored to a specific time scope, such as the length of a time series sample. The time scope clarifies exactly when the constraint takes effect and avoids impractical descriptions.

Combination with data facts. The unified design principles allow users to quickly check whether constraints meet their expectations. However, the value characteristics in constraints are diverse and unstructured, meaning not all such values can be directly supported by time series generation models. To address this issue and grounded in data facts, we further restrict constraint values into six types of data facts. For each type of data fact, we encode it using the proposed design principles and pair it with a corresponding icon to help users quickly recognize the constraint type and interpret its semantic meaning. For example, as shown in Fig. 7(b), the *aggregation* data fact defines a time interval with upper and lower bounds as well as a corresponding mean value; the *trend* constraint denotes that the change phase is either increasing or decreasing over the time interval. Notably, without the upper and lower bounds, the two horizontal lines would not be defined. More data facts constraint visualization can be accessed in the supplementary material. Furthermore, this set of design principles is not limited to data facts alone and can also be readily extended to support a broader range of constraint types in future applications.

C. Interactions of TimeShaper

We provide a set of interactions for evaluating the quality of the generated samples, exploration in fact-based time series data visualization, and examination and refinement in constraint visualization.

1) *Metric Monitoring*: To help users intuitively evaluate the quality of the generated samples, we adopt J-FTSD [41] as a distance metric. It measures the quality of generated time series samples, where lower values indicate closer alignment with the true distribution, and higher values reflect poorer quality. Accordingly, an increase signals worse quality relative to the previous iteration, prompting users to review the added constraints, while a decrease validates the effectiveness of the constraints and improvement of the generated sample quality.

2) *Data-Fact-Based Interactions*: Due to the large scale of generated time series samples, a single set of representative data facts is insufficient to capture enough details. To address this, we construct the time series samples into a hierarchy using hierarchical clustering [45]. For the time series samples

in each cluster, we use the proposed method to extract and visualize their representative data facts. However, limited space prevents displaying all data facts. To tackle this, we use a tree-cut algorithm to dynamically visualize data facts of interest. Based on the hierarchy, users can click a tree node to view its detailed information, while other nodes are collapsed to conserve space. This strategy enables our tool to scale to large datasets. For each node x in the hierarchy, we define its Degree of Interest (DOI) as:

$$\text{DOI}(x|y) = \text{API}(x) - \text{Dis}(x,y) \quad (5)$$

where $\text{API}(x)$ denotes the a priori importance of node x , measured by the total salience score of the data facts within node x . This importance helps identify underlying problematic data facts. $\text{Dis}(x,y)$ is the tree distance between node x and the focus node y selected by users. Initially, the focus node is the root node.

Users can click  to expand the node for detailed information. They can inspect the data facts and click them for further analysis. When users click on a data fact, it is highlighted with a dashed outline, and a bar chart and a line chart appear below. The bar chart shows the distribution of attributes of the data facts represented by the clicked one. To facilitate analysis, we discretized the continuous attributes into four categories based on quartiles, applied eta squared [53] to assess correlations across all attributes, and selected the top five most relevant ones. The attributes are sorted by correlation, from highest to lowest. Due to space limitation, the bar chart only shows categories whose cumulative percentage exceeds 50%, and darker color indicate a higher proportion. When users hover over a bar, its category is shown. The line chart displays the original/generated time series samples whose data facts are represented by the selected data fact. To reduce visual clutter, only ten samples are shown for both the original and generated time series samples. These two charts allow users to examine the raw samples and relevant attributes behind a data fact, helping them decide how to define proper constraints.

3) *Constraint-based Interactions*: To enable users to convert their ideas into actionable model constraints, we utilized an LLM (e.g., GPT-4 [50]) to parse user input, extract key semantic information, and transform it into structured constraints. Although the LLM is effective in parsing natural language, it still misinterprets user intent sometimes, and the users may want to refine the constraints during exploration. Therefore, we treat the parsed result as an initial constraint, and support users in examining and refining it before applying it to the generation process. In the constraint visualization, we provide several interactions to support constraint examination and refinement. In addition, users can click , , and  to directly add, delete, or edit the conditions and values of the constraints. They can also adjust the value range and time interval by dragging. During refinement, the violation ratio is updated accordingly, allowing users to observe how each edit changes the proportion of generated samples that violate the constraint. This feedback helps users identify more violating samples and refine the constraints more effectively.

VI. EVALUATION

To validate the effectiveness of the proposed TimeShaper in enhancing generative time series augmentation, we conducted a case study. Additionally, we performed quantitative evaluations to demonstrate that our method effectively improves the downstream model performance.

Datasets. We conducted the case study and experiments on two real-world datasets: Traffic [54] and Air Quality [55]. The Traffic dataset contains single-channel hourly traffic volume recordings from 2012 to 2018, along with seven discrete attributes (year, month, day, hour, holiday, board weather descriptions, and fine weather descriptions) and four continuous attributes (temperature, rainfall, snowfall, and cloud conditions). The Air Quality dataset records hourly air pollution data from 12 monitoring stations across Beijing between 2013 and 2017. It includes six key pollutant channels, namely PM2.5, PM10, O3, NO2, SO2, and CO, as well as six discrete attributes (year, month, day, hour, wind direction, and monitoring station) and five continuous attributes (rainfall, temperature, pressure, dew point temperature, and wind speed). Both datasets are partitioned using a 96-hour sliding window with a stride of 24 hours. After 0-1 data normalization and train-test splitting, we obtain 1,449 training and 201 testing samples for the Traffic dataset, and 12,643 training and 1,580 testing samples for the Air Quality dataset.

A. Case Study

To demonstrate the effectiveness of TimeShaper in generative time series data augmentation, we conducted a case study with E1, one of the experts in requirement analysis. This case study was conducted on the Traffic and Air Quality datasets. E1 aimed to improve the classification accuracy on both datasets using Time Weaver [41], one of the state-of-the-art automatic generative data augmentation methods, to augment the training data. However, he observed that model performance degraded as the number of augmented samples increased. Therefore, he chose to use our tool for diagnosis. In the case study, we used the pair analytics protocol [56], which enables experts to focus on analytical tasks while we handle the navigation of the tool.

1) Interactively Augmenting the Traffic Dataset: Initially, E1 had no prior knowledge about the Traffic dataset. Therefore, he used all the attributes as the conditions and without specifying values in the constraints to generate $1 \times$ samples. These generated samples, along with the original ones, were input into our tool for analysis.

Overview. E1 began the analysis from the fact-based time series data visualization. In this visualization, the original and generated samples are clustered in a hierarchy. He noticed that in many clusters, the data facts with the longest bars exhibited a highly noticeable difference in color intensity between the original and generated samples (e.g., Figs. 8A₁ and 8A₂). E1 was curious about this and wanted to choose one of the clusters to analyze. Considering that the root node can typically reflect global information, E1 chose to analyze the root node.

Identifying out-of-scope traffic volumes. After expanding the root node, E1 found that the data facts with the longest bars of different color intensity correspond to the extreme type. The color of *extreme* data facts in the original samples is very light, whereas the color in the generated samples is very dark (Fig. 8A₃). Curious about the large color difference, he clicked the *extreme* data fact in the generated samples. The corresponding original and generated time series samples of the chosen data fact were presented below. E1 observed an obvious out-of-scope value in the generated samples: the minimum value of one time series was less than 0 (Fig. 8B). This reminded him that time series data are typically normalized to the range [0, 1] during data preprocessing. This indicated that the generation process had produced values outside the expected numerical range. E1 inferred that the generative model may not have fully learned the data distribution of the original samples, resulting in such out-of-scope samples. Therefore, he turned to the chat view to express his ideas to define constraints.

In the chat view, E1 entered “All values must be between 0 and 1” (Fig. 8C). Then, an *aggregation* constraint enforcing a maximum value of 1 and a minimum value of 0 was automatically generated in the constraint view (Fig. 8D). He examined the bar chart on the left, found that 3% of the data violated the constraint (Fig. 8E), which is consistent with

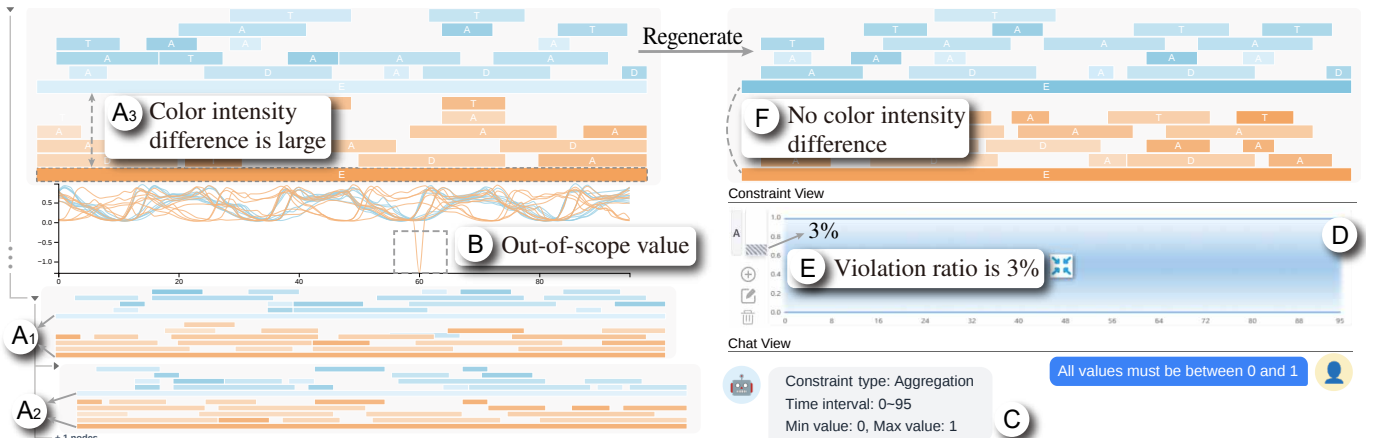


Fig. 8. Identifying and correcting out-of-scope traffic volumes through fact-based exploration and valid-range constraint refinement.

earlier observations. Therefore, E1 verified the constraint and applied it to the data generation process for regeneration.

After the regeneration, E1 found that there is no longer a color density difference in the *extreme* data fact (Fig. 8F). Meanwhile, he noticed that in the constraint view, the proportion of constraint-violating data has decreased from 3% to 0%, which means that the newly generated samples are all in the scope (Fig. 5F₁). Moreover, the distance metric decreased (Fig. 5H₁). All of these patterns indicated an increase in the quality of the generated samples. Therefore, E1 thought this constraint was useful and accepted it.

Identifying abnormal traffic volumes across different time periods. After the regeneration, the hierarchy is updated, E1 found some difference existed in the data facts of the original and generated samples in many clusters. He decided to analyze them one by one. He first analyzed the data facts in one cluster (Fig. 5A), because this cluster is recommended by the tree cut algorithm. In this cluster, a dark-colored *trend* data fact that appeared in the original samples but was absent in the generated samples (Fig. 5B). By clicking this fact, E1 examined its corresponding samples, and he found that the original samples clearly exhibited an upward trend within the time interval of this fact (the blue lines), whereas this trend was not captured in the generated samples (the orange lines, Fig. 5C). Beyond this time interval, E1 also identified obvious differences in the change phase between the generated samples and the original samples. Similar patterns are also found in other data facts, such as the *aggregation* data facts in Fig. 5D₁. E1 thought that these issues are all related to time, so he checks the hour attribute of each *aggregation* data fact (Fig. 5D₂). E1 observed that the traffic volume in the generated samples was higher than that in the original samples during late-night and dawn hours, which was clearly abnormal. On the contrary, during the same daytime period, the original samples showed higher value in the morning, which met his expectation, whereas the generated samples exhibited lower value in the afternoon.

Based on these findings, E1 concluded that the traffic volume during the daytime should be higher than at nighttime. Therefore, E1 decided to add constraints to minimize deviations. To this end, he defined the time scope of the constraint as 24 hours and entered the instruction “The daytime values should be greater than nighttime values,” and then got a rank constraint (Fig. 5E). The rank constraint includes two time intervals, representing two different time periods. Each time interval had an order that indicated the relationship between its values. However, E1 found that these intervals did not meet his expectation: daytime and nighttime had been set to 12–23 and 0–11, respectively, whereas daytime should be 6–17 and nighttime should be 18–23 and 0–5. Therefore, he displayed the constraint and decided to refine it. By dragging the time intervals, E1 first adjusted the daytime and nighttime intervals to the correct ranges (Fig. 5F₂). He then clicked the ⊕ icon to add a new nighttime interval, 18–23, and set its rank as 2. With these operations, the constraint was correctly refined. After refinement, the left bar chart showed that 70% of the data violated this refined constraint (Fig. 5F₃), indicating that

up to 70% of the generated samples were inconsistent with real-world conditions. Thus, after verifying that this constraint was correct, E1 added this constraint in the generation process and regenerated a new round of data.

After regeneration, E1 found that the difference between the data facts of the original and generated samples decreased. For example, the *trend* data fact missing in the previous round now appeared (Fig. 5G₁), and the color difference between the *aggregation* data facts decreased (Fig. 5G₂). Compared with the previous round, the change phase of the generated samples with this constraint added is significantly closer to that of the original samples (e.g., Fig. 5G₁). Furthermore, the distance metric has decreased significantly (Fig. 5H₂). However, E1 noticed that although most of the generated samples satisfied the newly added constraint, a small portion (30%) still violated it (Fig. 5F₄). E1 was confused by this and consulted E2, who had rich experience in traffic-related data. E2 explained that occasional violations of this constraint are reasonable because real-world scenarios can involve various unexpected events (e.g., large-scale nighttime events or daytime traffic control), which can easily lead to lower traffic volume during the daytime than at night. Thus, E1 felt that the change to the constraint was meaningful and accepted it.

E1 also checked other data facts and defined new constraints to improve the generated time series sample quality. In total, E1 added 6 constraints.

2) *Interactively Augmenting the Air Quality Dataset:* After improving the data quality of the generated samples of the Traffic dataset, E1 then worked on enhancing generated samples of the Air Quality dataset. Considering that PM2.5 poses the most significant harm to human health and the ecological environment in real-world scenarios [57], E1 focused on PM2.5 in the case study. Similarly, E1 used all the attributes as the conditions and without specifying values in the constraints to generate $1 \times$ samples.

Overview. The original and generated samples were clustered into a hierarchy and displayed in the fact-based time series data visualization (Fig. 9(a)). In this hierarchy, the cluster recommended by the tree cut algorithm caught E1’s attention. In this cluster, E1 observed color intensity differences between the data facts from the original and generated samples. He first analyzed the pair of *aggregation* data facts because they had an obvious difference in color density and covered a long time interval (Fig. 9A₁). After clicking on this generated data fact, E1 found its corresponding time series samples to be relatively scattered, with no obvious temporal trend (Fig. 9B). Therefore, he shifted his focus to time-independent variable attributes for further analysis. Dew point temperature was ordered first, which meant the highest correlation with PM2.5, so E1 focused on it (Fig. 9C₁).

Identifying inverse correlations. E1 examined the dew point temperature on the data facts of the original and generated samples, respectively. In generated samples, low dew point temperature is associated with high PM2.5 value (Fig. 9C₁), but in original samples, high dew point temperature corresponds to high PM2.5 value (Fig. 9C₂). In reality, high dew point temperature suppresses pollutant dispersion and

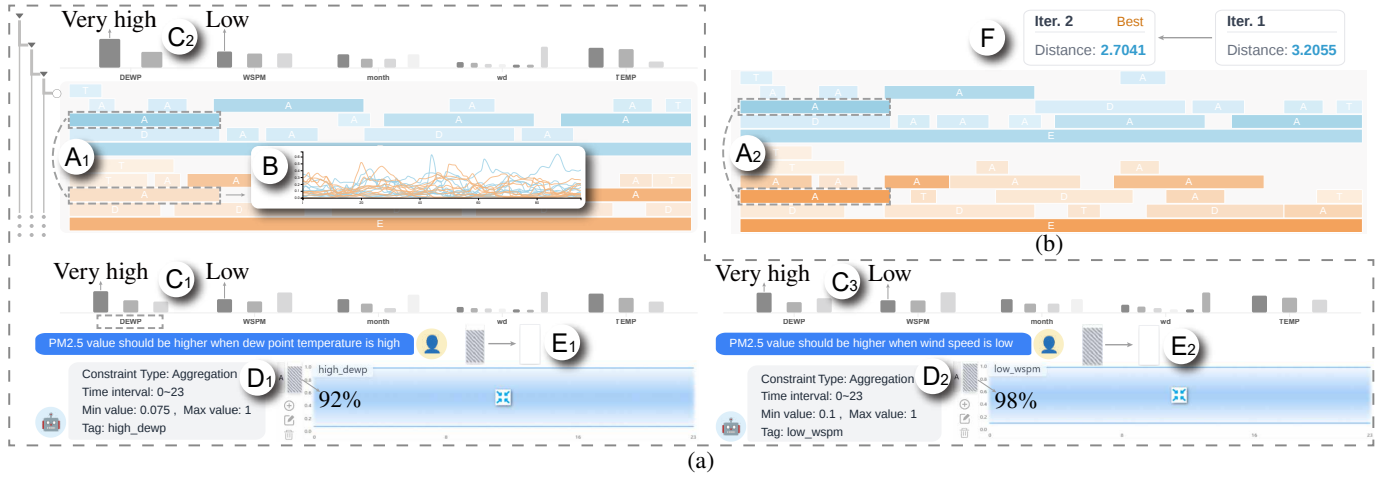


Fig. 9. Fact-based comparison and constraint visualization for improving air-quality generation: (a) unconstrained generation before applying the constraints; (b) regeneration after imposing the specified constraints.

raises PM2.5 levels, which the generated samples failed to reflect. Accordingly, E1 sets the constraint as “PM2.5 value should be higher when dew point temperature is high.” Then, he examined the corresponding constraint in the constraint view and found that the violation ratio of this constraint in the generated samples reached 92% (Fig. 9D₁), which was highly surprising. Therefore, E1 saved this constraint to be incorporated into the generation process later.

Following the same way of dew point temperature, E1 checked the wind speed, which had the second-highest correlation, in both the original and generated samples. In the original samples, the wind speed was relatively low, and the PM2.5 value was high, which met his expectations (Fig. 9C₂). However, in the generated samples, the wind speed was relatively low, and the PM2.5 value was also low (Fig. 9C₁). Intuitively, higher wind speed favors the dispersion of PM2.5, leading to lower values. Thus, E1 set “PM2.5 value should be higher when the wind speed is low.” as the second constraint. Upon examining the constraint, he found that the violation ratio of this constraint also reached 98% (Fig. 5D₂). This further confirmed that the generated samples failed to capture realistic physical relationships between meteorological conditions and PM2.5 concentration. After confirming that this constraint had also been set correctly, E1 decided to incorporate both constraints into the data generation process and generated a new round of data.

After regeneration, E1 observed that the *aggregation* data fact of the generated samples appeared with a darker color (Fig. 9A₂). He verified the data facts and found that both corresponded to high dew point temperature and low wind speed with high PM2.5 value (Fig. 9C₃), which was consistent with the two constraints set in the previous step. When inspecting the violation of the constraints, E1 noticed that the violation ratios of these two constraints had dropped significantly — from 92% and 98% to 0% respectively (Figs. 9E₁ and 9E₂). Moreover, the distance metric also decreased a lot (Fig. 9F). Accordingly, with the two constraints, the quality of the generated samples improved. E1 thought that the constraints

were effective and accepted them.

E1 also checked other data facts and defined new constraints to improve the generated time series sample quality. In total, E1 added 5 constraints.

B. Quantitative Evaluation

We conducted quantitative evaluations in two different downstream tasks, classification and missing value imputation, to demonstrate the effectiveness of our proposed methods.

Outperform the automatic generative data augmentation method. We employed a time series classification task for evaluation. Following the setup of Time Weaver [41], we used a standard ResNet 1D model [58] for training. For the Traffic dataset, we conducted multi-class classification on the board weather description attribute; for the Air Quality dataset, we performed single-label classification on the station attribute. Classification performance was measured using accuracy. Both our method and the automatic method generated augmented samples at different multipliers, which were then combined with the original samples for training. All models were evaluated on a held-out test set. As shown in Table I, the accuracy of the automatic method drops significantly when the number of generated samples is less than $5\times$ that of the original samples, and this decline is caused by the low-quality generated samples. However, our method mitigates this decline and consistently outperforms the automatic method on both datasets across different multipliers, ultimately achieving improvements of approximately 3% and 14%, respectively. Notably, on the Air Quality dataset, our method significantly improves the accuracy over the original-data baseline by nearly 30%, demonstrating that the refined generated samples can substantially benefit downstream model training. This improvement stems mainly from our method’s ability to effectively identify low-quality generated samples and impose constraints to generate higher-quality samples.

Generalization of our method. Beyond classification, time series data also play a critical role in many other tasks, such as imputation. Our proposed method can be readily extended

TABLE I
COMPARISON OF CLASSIFICATION ACCURACY (%) OF DIFFERENT METHODS. THE BEST RESULT IN EACH SETTING IS IN **BOLD**.

Method	Original	+1×	+5×	+10×	+20×	+30×
Traffic						
Baseline	81.17	—	—	—	—	—
Automatic	—	76.02	79.77	82.02	82.82	82.82
Ours	—	80.82	80.62	84.02	85.41	85.61
Air Quality						
Baseline	37.63	—	—	—	—	—
Automatic	—	28.34	34.66	38.27	45.29	51.93
Ours	—	44.85	47.38	42.76	58.76	66.03

to these related tasks to enrich the training dataset and further enhance model performance. To verify this, we apply the generative samples to the time series imputation task. We first employ TimesNet [59], a representative time series model, to evaluate the imputation performance. Both our method and the automatic method generated 30× samples and combined with the original samples for training. Following common practice, we randomly mask 25% of the timestamps in each time series before training. After training, we reported two standard evaluation metrics, MSE and MAE, with lower values indicating better imputation quality. The results are presented in Table II. Our method achieves the best performance. Although the automatic method provides significant improvement, our method can achieve further performance gains. These results demonstrate that our method has strong generalization ability to other time series tasks and effectively improves downstream model performance.

VII. EXPERT FEEDBACK AND DISCUSSION

Following the case study, we conducted four expert interviews to collect feedback. These interviews involved two experts (E1, E2) who had participated in our requirement analysis and two additional invited experts (E5, E6). Both the newly invited experts were Ph.D. students who had over two years of experience in time series data analysis. Each interview lasted between 30 and 50 minutes. The experts provided positive feedback on the usability of TimeShaper, while also identifying notable limitations and suggesting constructive directions for future research.

A. Usability

Efficient exploration of time series data. All experts appreciated that the fact-based time series data visualization helped them efficiently obtain an overview of generated samples. E1 said, “Compared with examining individual time series as line charts, the representative data facts provide me with an overview of generated samples and help me quickly identify important patterns with less effort.” Meanwhile, E6 highlighted the practicality of the co-layout design for comparing original and generated facts. He noted that, because data facts of the same type and similar time intervals are placed in aligned positions, he could directly compare corresponding facts without repeatedly searching across the two charts. He

TABLE II
COMPARISON OF METHODS FOR MISSING VALUE IMPUTATION. THE BEST RESULT IN EACH SETTING IS IN **BOLD**.

Method	Traffic		Air Quality	
	MSE	MAE	MSE	MAE
Original	0.2077	0.3308	0.1557	0.2485
Automatic	0.0815	0.2002	0.0900	0.1760
Ours	0.0738	0.1913	0.0847	0.1704

said, “During the exploration, I can quickly check whether a pattern is missing or changed in value by comparing the aligned regions of the original and generated visualizations.”

Intuitive constraint definition and refinement. All experts found this definition-and-refinement workflow intuitive and useful. E5 commented that the LLM-based parser helped quickly define initial constraints from natural language instead of manually writing rules or regular expressions, while the constraint visualization and provided interactions made the constraints easy to examine and refine. For example, users could refine an incorrect constraint by dragging the time interval or directly editing its condition or value, with the violation ratio updated in real time. E1 stated, “The text generated by the LLM tells me what the constraint is, but the constraint visualization helps me check whether it is applied in the right place and whether the violation ratio makes sense.”

Generalization to other time series downstream tasks. This work focuses on improving the quality of generated time series data to construct better training datasets for downstream models. Therefore, the proposed method is not tied to a specific downstream task. Our evaluation demonstrates that the proposed method improves downstream model performance in both classification and imputation tasks. Experts further suggested that the same workflow could benefit other time series tasks that rely on high-quality training data, such as forecasting and anomaly detection, by helping users identify low-quality generated samples and refine generation constraints.

B. Limitations and Future Work

Simultaneous analysis and constraints of multiple variables. Currently, the visual analysis system primarily focuses on analyzing and constraining a single variable at a time, and provides limited support for coordinated visualization, joint analysis, and constraint definition across multiple variables. When users need to explore complex relationships or interdependencies among multiple variables, such as correlations between PM2.5 and meteorological attributes, the current design can help reveal problematic patterns through attribute-based inspection, but does not yet support for expressing such relationships as explicit multi-variable constraints. For example, constraints such as “PM2.5 and temperature should be negatively correlated” are beyond the six single-variable data fact types currently supported. Therefore, extending the constraint visualization and interaction design to support multi-variable relationship constraints is an important and promising direction for future work.

Support for irregular and variable-length time series. In our current implementation, we focus on regularly sampled, fixed-length time series, where all samples share the same timestamp sequence. However, in many real-world scenarios, time series can be irregularly sampled or variable-length. For example, event-based time series (e.g., medical monitoring) often exhibit irregular timestamps and varying sequence lengths. Our pipeline could be extended to such data by first aligning or segmenting sequences into comparable temporal units, or by redefining data facts over event-based intervals rather than fixed timestamp indices. Nevertheless, irregular and variable-length data would require redesigning the representative fact extraction, co-layout method and corresponding constraint visualization. How to support such heterogeneous time series data remains an important direction for future work.

VIII. CONCLUSION

In conclusion, this paper presents a novel data-fact-driven interactive generative method for augmenting time series samples. By employing a representative data fact extraction method and a data fact co-layout method, users can effectively and efficiently explore both original and generated time series samples, thereby identifying low-quality generated samples. Additionally, the proposed constraint visualization and unified design principle ease the process of validating and refining constraints. The effectiveness of the proposed methods is demonstrated through quantitative evaluation and a case study.

REFERENCES

- [1] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, "Financial time series forecasting with deep learning: A systematic literature review: 2005–2019," *Applied Soft Computing*, vol. 90, p. 106181, 2020.
- [2] B. Shickel, P. J. Tighe, A. Bihorac, and P. Rashidi, "Deep EHR: a survey of recent advances in deep learning techniques for electronic health record (ehr) analysis," *IEEE Journal of Biomedical and Health Informatics*, vol. 22, no. 5, pp. 1589–1604, 2017.
- [3] R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, and R. X. Gao, "Deep learning and its applications to machine health monitoring," *Mechanical Systems and Signal Processing*, vol. 115, pp. 213–237, 2019.
- [4] X. Shi, S. Wang, Y. Nie, D. Li, Z. Ye, Q. Wen, and M. Jin, "Time-MoE: Billion-scale time series foundation models with mixture of experts," in *Proceedings of the International Conference on Learning Representations*, 2025.
- [5] C. Bastidon, A. Parent, P. Abry, P. Borgnat, P. Jensen, and B. Pascal, "Detecting global financial crises with scarce data by multivariate nonlinear filtering," *Journal of Physics: Complexity*, vol. 6, no. 3, p. 035003, 2025.
- [6] S. S. Narasimhan, S. Agarwal, L. Rout, S. Shakkottai, and S. P. Chinchali, "Constrained posterior sampling: Time series generation with hard constraints," in *Proceedings of the Conference on Neural Information Processing Systems*, 2025.
- [7] Y. Zhang, D. Zhou, B. Hooi, K. Wang, and J. Feng, "Expanding small-scale datasets with guided imagination," in *Proceedings of the Advances in Neural Information Processing Systems*, 2023, pp. 76558–76618.
- [8] C. Chen, F. Lv, Y. Guan, P. Wang, S. Yu, Y. Zhang, and Z. Tang, "Human-guided image generation for expanding small-scale training image datasets," *IEEE Transactions on Visualization and Computer Graphics*, 2025.
- [9] Y. Wang, Z. Sun, H. Zhang, W. Cui, K. Xu, X. Ma, and D. Zhang, "DataShot: Automatic generation of fact sheets from tabular data," *IEEE Transactions on Visualization and Computer Graphics*, vol. 26, no. 1, pp. 895–905, 2019.
- [10] J. Zhao, F. Chevalier, and R. Balakrishnan, "KronoMiner: using multi-foci navigation for the visual exploration of time-series data," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2011, pp. 1737–1746.
- [11] M. Sips, P. Köthür, A. Unger, H.-C. Hege, and D. Dransch, "A visual analytics approach to multiscale exploration of environmental time series," *IEEE Transactions on Visualization and Computer Graphics*, vol. 18, no. 12, pp. 2899–2907, 2012.
- [12] T. Fujiwara, N. Sakamoto, J. Nonaka, K. Yamamoto, K.-L. Ma *et al.*, "A visual analytics framework for reviewing multivariate time-series data with dimensionality reduction," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, pp. 1601–1611, 2020.
- [13] M. Emmei, N. Okami, T. Fujiwara, N. Sakamoto, and J. Nonaka, "Visual analytics for multivariate time-series data using interactive dimensionality reduction methods," in *Proceedings of the IEEE Pacific Visualization Conference*, 2025, pp. 392–398.
- [14] Y. Yu, D. Kruffy, J. Jiao, T. Becker, and M. Behrisch, "PSEUDO: Interactive pattern search in multivariate time series with locality-sensitive hashing and relevance feedback," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 1, pp. 33–42, 2022.
- [15] J. Bernard, C.-M. Barth, E. Cuba, A. Meier, Y. Peiris, and B. Shneiderman, "IVESA—visual analysis of time-stamped event sequences," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 4, pp. 2235–2256, 2024.
- [16] Y. Shi, B. Chen, Y. Chen, Z. Jin, K. Xu, X. Jiao, T. Gao, and N. Cao, "Supporting guided exploratory visual analysis on time series data with reinforcement learning," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 1, pp. 1172–1182, 2023.
- [17] S. Liu, Y. Tian, Z. Deng, W. Cui, H. Zhang, D. Weng, and Y. Wu, "Relation-driven query of multiple time series," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 8, pp. 4210–4225, 2025.
- [18] L. Meng, S. Liu, K. Yang, J. Xu, Z. Deng, D. Weng, and Y. Wu, "ChronoDeck: a visual analytics approach for hierarchical time series analysis," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 12, pp. 10488–10502, 2025.
- [19] G. Y.-Y. Chan, L. G. Nonato, T. Palpanas, C. T. Silva, and J. Freire, "TiVy: Time series visual summary for scalable visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 32, no. 1, pp. 232–242, 2025.
- [20] Y.-C. Huang, J. Chen, D. Liu, and K.-L. Ma, "SigTime: Learning and visually explaining time series signatures," *IEEE Transactions on Visualization and Computer Graphics*, vol. 32, no. 2, pp. 2099–2113, 2025.
- [21] K. Xu, J. Yuan, Y. Wang, C. Silva, and E. Bertini, "mTSeer: Interactive visual exploration of models on multivariate time-series forecast," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2021, pp. 1–15.
- [22] X. Yi, Y. Yang, Y. Song, B. Zhou, C. Li, S. Liu, D. Deng, D. Weng, and Y. Wu, "A domain knowledge-driven visual analytics system for photovoltaic power time series forecasting," *Journal of Visualization*, vol. 28, no. 4, pp. 895–915, 2025.
- [23] J. Hao, Q. Shi, Y. Ye, and W. Zeng, "TimeTuner: diagnosing time representations for time-series forecasting with counterfactual explanations," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 1, pp. 1183–1193, 2023.
- [24] C. Lv, K. Ren, H. Zhang, J. Fu, and Y. Lin, "PEVis: visual analytics of potential anomaly pattern evolution for temporal multivariate data," *Journal of Visualization*, vol. 25, no. 3, pp. 575–591, 2022.
- [25] K. S. Omar, S. Wang, R. Kungumaraju, T. Bhatt, and F. Miranda, "VIGMA: An open-access framework for visual gait and motion analytics," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 10, pp. 8143–8158, 2025.
- [26] H. Janetzko, F. Stoffel, S. Mittelstädt, and D. A. Keim, "Anomaly detection for visual analytics of power consumption data," *Computers & Graphics*, vol. 38, pp. 27–37, 2014.
- [27] N. Cao, C. Lin, Q. Zhu, Y.-R. Lin, X. Teng, and X. Wen, "Voila: Visual anomaly detection and monitoring with streaming spatiotemporal data," *IEEE Transactions on Visualization and Computer Graphics*, vol. 24, no. 1, pp. 23–33, 2017.
- [28] M. C. Hao, U. Dayal, D. A. Keim, R. K. Sharma, and A. Mehta, "Visual analytics of anomaly detection in large data streams," in *Proceedings of the Visualization and Data Analysis*, vol. 7243, 2009, pp. 84–93.
- [29] Z. Deng, D. Weng, X. Xie, J. Bao, Y. Zheng, M. Xu, W. Chen, and Y. Wu, "Compass: Towards better causal analysis of urban time series," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 1, pp. 1051–1061, 2021.
- [30] D. Liu, S. Alnegheimish, A. Zytek, and K. Veeramachaneni, "MTV: Visual analytics for detecting, investigating, and annotating anomalies in multivariate time series," *Proceedings of the ACM on Human-Computer Interaction*, vol. 6, no. CSCW1, pp. 1–30, 2022.

- [31] J. He, X. Wang, K. K. Wong, X. Huang, C. Chen, Z. Chen, F. Wang, M. Zhu, and H. Qu, "VideoPro: A visual analytics approach for interactive video programming," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 1, pp. 87–97, 2023.
- [32] C. Chen, J. Chen, W. Yang, H. Wang, J. Knittel, X. Zhao, S. Koch, T. Ertl, and S. Liu, "Enhancing single-frame supervision for better temporal action localization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 6, pp. 2903–2915, 2024.
- [33] T. Tang, Y. Wu, Y. Wu, L. Yu, and Y. Li, "VideoModerator: A risk-aware framework for multimodal video moderation in e-commerce," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 1, pp. 846–856, 2021.
- [34] T. Gschwandtner, W. Aigner, S. Miksch, J. Gärtner, S. Kriglstein, M. Pohl, and N. Suchy, "TimeCleanser: A visual analytics approach for data cleansing of time-oriented data," in *Proceedings of the International Conference on Knowledge Technologies and Data-Driven Business*, 2014, pp. 1–8.
- [35] C. Arbesser, F. Spechtenhauser, T. Mühlbacher, and H. Piringer, "Vis-plause: Visual data quality assessment of many time series using plausibility checks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 23, no. 1, pp. 641–650, 2016.
- [36] H. Yeon, H. Son, and Y. Jang, "Visual imputation analytics for missing time-series data in bayesian network," in *IEEE International Conference on Big Data and Smart Computing*, 2020, pp. 303–310.
- [37] T. Gschwandtner and O. Erhart, "Know your enemy: Identifying quality problems of time series data," in *Proceedings of the IEEE Pacific Visualization Symposium*, 2018, pp. 205–214.
- [38] J. Bernard, M. Hutter, H. Reinemuth, H. Pfeifer, C. Bors, and J. Kohlhammer, "Visual-interactive preprocessing of multivariate time series data," *Computer Graphics Forum*, vol. 38, no. 3, pp. 401–412, 2019.
- [39] Z. Kong, W. P. and Jiaji Huang, K. Zhao, and B. Catanzaro, "DiffWave: A versatile diffusion model for audio synthesis," in *Proceedings of the International Conference on Learning Representations*, 2021.
- [40] X. Yuan and Y. Qiao, "Diffusion-TS: Interpretable diffusion for general time series generation," in *Proceedings of the International Conference on Learning Representations*, 2024.
- [41] S. S. Narasimhan, S. Agarwal, O. Akcin, S. Sanghavi, and S. Chinchali, "Time Weaver: a conditional time series generation model," in *Proceedings of the International Conference on Machine Learning*, 2024.
- [42] Y. Li, H. Meng, Z. Bi, I. T. Urnes, and H. Chen, "Population aware diffusion for time series generation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 17, pp. 18 520–18 529, 2025.
- [43] I. Naiman, N. Berman, I. Pemper, I. Arbiv, G. Fadlon, and O. Azencot, "Utilizing image transforms and diffusion models for generative modeling of short and long time series," in *Proceedings of the Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 121 699–121 730.
- [44] T. Gonen, I. Pemper, I. Naiman, N. Berman, and O. Azencot, "Time series generation under data scarcity: A unified generative modeling approach," in *Proceedings of the Conference on Neural Information Processing Systems*, 2025.
- [45] C. K. Reddy and B. Vinzamuri, "A survey of partitional and hierarchical clustering algorithms," in *Data clustering*. Chapman and Hall/CRC, 2018, pp. 87–110.
- [46] W. Javed, B. McDonnell, and N. Elmquist, "Graphical perception of multiple time series," *IEEE Transactions on Visualization and Computer Graphics*, vol. 16, no. 6, pp. 927–934, 2010.
- [47] U. Feige, "A threshold of $\ln n$ for approximating set cover," *Journal of the ACM*, vol. 45, no. 4, pp. 634–652, 1998.
- [48] L. Chittaro and C. Combi, "Representation of temporal intervals and relations: information visualization aspects and their evaluation," in *Proceedings of the International Symposium on Temporal Representation and Reasoning*, 2001, pp. 13–20.
- [49] E. G. Coffman Jr, M. R. Garey, and D. S. Johnson, "Approximation algorithms for bin-packing—an updated survey," in *Algorithm design for computer system design*. Springer, 1984, pp. 49–106.
- [50] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv: 2303.08774*, 2023.
- [51] X. Zhu, M. A. Nacenta, Ö. Akgün, and P. Nightingale, "How people visually represent discrete constraint problems," *IEEE Transactions on Visualization and Computer Graphics*, vol. 26, no. 8, pp. 2603–2619, 2019.
- [52] A. Karagappa, P. K. Betz, J. Gilg, M. Zeumer, A. Gerndt, and B. Preim, "Enhancing uncertainty communication in time series predictions: Insights and recommendations," *arXiv preprint arXiv:2408.12365*, 2024.
- [53] J. T. Richardson, "Eta squared and partial eta squared as measures of effect size in educational research," *Educational Research Review*, vol. 6, no. 2, pp. 135–147, 2011.
- [54] J. Hogue, "Metro Interstate Traffic Volume," UCI Machine Learning Repository, 2019, DOI: <https://doi.org/10.24432/C5X60B>.
- [55] S. Chen, "Beijing Multi-Site Air Quality," UCI Machine Learning Repository, 2017, DOI: <https://doi.org/10.24432/C5RK5G>.
- [56] R. Arias-Hernandez, L. T. Kaastra, T. M. Green, and B. Fisher, "Pair analytics: Capturing reasoning processes in collaborative visual analytics," in *IEEE Hawaii International Conference on System Sciences*, 2011, pp. 1–10.
- [57] A. J. Cohen, M. Brauer, R. Burnett, H. R. Anderson, J. Frostad, K. Estep, K. Balakrishnan, B. Brunekreef, L. Dandona, R. Dandona *et al.*, "Estimates and 25-year trends of the global burden of disease attributable to ambient air pollution: an analysis of data from the global burden of diseases study 2015," *The lancet*, vol. 389, no. 10082, pp. 1907–1918, 2017.
- [58] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proceedings of the European conference on computer vision*, 2016, pp. 630–645.
- [59] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2d-variation modeling for general time series analysis," in *Proceedings of the International Conference on Learning Representations*, 2023.



Fei Lyu received the B.S. degree in software engineering from Hunan University in 2023. He is currently pursuing a Ph.D. degree in computer science at Hunan University. His research interests include visual analytics and generative models.



Guangze Deng is an undergraduate student at the College of Computer Science and Electronic Engineering, Hunan University, where he is working toward his B.S. degree in computer science (expected 2027). His research interests include data visualization, visual analytics, and human-AI interaction.



Zhuo Tang received the Ph.D. in computer science from Huazhong University of Science and Technology, China, in 2008. He is currently a professor of the College of Computer Science and Electronic Engineering at Hunan University. He is also the chief engineer of the National Supercomputing Center in Changsha. His majors are distributed computing system, cloud computing, and parallel processing for big data, including distributed machine learning, security model, parallel algorithms, and resources scheduling and management in these areas. He has

published almost 230 journal articles and book chapters. He is a member of IEEE/ACM and CCF.



Kenli Li received the Ph.D. degree in computer science from Huazhong University of Science and Technology, China, in 2003. His major research areas include high performance computing, parallel computing, grid and cloud computing. He has published more than 300 research papers in international conferences and journals such as IEEE-TC, IEEE-TKDE, IEEE-TPDS, and ICDE. He is currently or has served on the editorial boards of the IEEE-TC, IEEE-TSUSC, and IEEE-TII. He is a senior member of IEEE and an outstanding member of CCF.



Changjian Chen received the B.S. degree from the University of Science and Technology of China and the Ph.D. degree from Tsinghua University. He is currently an associate professor at Hunan University. His research interests include visual analytics and machine learning, with a particular focus on visual analysis methods for improving training data quality.